

Contemporary C++: *Learning Modern C++ in a Modern Way*



مدرس: سعید امراللہی بیوکی

Saeed Amrollahi Boyouki

www.Linkedin.com/in/saeed-amrollahi-boyouki

Agenda 25/30

Session 25. More on Memory management operators

- Allocation and deallocation operators
- Structs vs. Classes
- References and further readings
- Q&A

120 min (incl. Q & A)



Allocation and deallocation operators

Allocation and deallocation operators

- Operator new acquires its memory by calling an operator new(). Similarly, operator delete frees its memory by calling an operator delete().

Allocation and deallocation operators

- Operator new acquires its memory by calling an operator new(). Similarly, operator delete() frees its memory by calling an operator delete().

```
void* operator new(size_t); // use for individual object  
void* operator new[](size_t); // use for array  
void operator delete(void*, size_t); // use for individual object  
void operator delete[](void*, size_t); // use for array
```

Allocation and deallocation operators

- Operator new acquires its memory by calling an operator new(). Similarly, operator delete() frees its memory by calling an operator delete().

```
void* operator new(size_t); // use for individual object  
void* operator new[](size_t); // use for array  
void operator delete(void*, size_t); // use for individual object  
void operator delete[](void*, size_t); // use for array
```

- The new expression vs. the new operator
- The C generic pointer: the void*
- The void *: Pointer to some memory that the compiler doesn't know the type of
- Placement new operators

Memory management: summary

- Allocation functions:

new T

Allocates and default-initializes a new object of type T and returns a pointer to the object.

new T(args)

Allocates and initializes a new object of type T using *args* to initialize the object.
Returns a pointer to the object.

new T[n]

Allocates and default-initializes an array of n new objects of type T. Return a pointer to the initial element in the array.

- De-allocation functions:

delete p

Destroys the object to which p points and frees the memory used to hold *p. The pointer must point at an object that was dynamically allocated.

delete [] p

Destroys the objects in the array to which p points and frees the memory used to hold the array. The pointer must point to the initial element of an array that was dynamically allocated.

new

↔

delete

new []

↔

delete []

New and delete operators



Using New And Delete Operators *Prog.*

struct vs. class

- By definition a struct is a class in which members are by default public.

```
struct s { ...
```

```
class s { public: ...
```

```
class Point {  
    int x, y;  
public:  
    Point(int xx, int yy) : x{xx}, y{yy} {}  
    // ...  
};
```

≡

```
struct Point {  
    private:  
        int x, y;  
    public:  
        Point(int xx, int yy) : x{xx}, y{yy} {}  
        // ...  
};
```

- Use struct for classes that have all data public.
- Use struct for not quite proper types; just data structures.

Q *What's the difference between the keywords struct and class?*

A Emotionally, most developers make a strong distinction between a class and a struct. A struct simply *feels* like an open pile of bits with very little in the way of encapsulation or functionality. A class *feels* like a living and responsible member of society with intelligent services, a strong encapsulation barrier, and a well defined interface.

C++-FAN

R eferences and further readings

[Str13] Bjarne Stroustrup. The C++ Programming Language. 4th edition. Addison Wesley. Reading Mass. USA. May 2013.

Thanks for your patience ...

A man who asks a question is a fool for minute,
The man who does not ask, is a fool for a life.
- Confucius

Learning to ask the right (often hard) questions is an essential part of learning to think as a programmer.

- Bjarne Stroustrup *programming Principles and Practice Using C++, page 4.*

There is no stupid question, but there is stupid answer.
- Howard Hinnant

